

Touying 文法リファレンス  
テキスト・画像・コード・レイアウト 完全網羅・2026 年

## テキスト書式

テキスト書式

太字 (bold)、イタリック、イン  
ラインコード

下線テキスト、~~取り消し線~~

上付き:  $x^2$ 、下付き:  $H_2O$

赤色テキスト、大きい青

ハイライト

**\*太字\***、*\_イタリック\_*、`インライン  
コード`

`#underline`[下線]、`#strike`[取り  
消し線]

`x#super`[2]、`H#sub`[2]0

`#text(fill: red)`[赤色テキスト]  
`#text(fill: blue, size:`  
`1.3em)`[大きい]

#highlight[ハイライト]

# 見出し・リスト

見出し・リスト

## 箇条書き (unordered)

- 項目 A
  - ▶ ネスト A-1
  - ▶ ネスト A-2
- 項目 B

## 番号付き (ordered)

1. ステップ 1
2. ステップ 2

- 項目 A
    - ネスト A-1
  - 項目 B
- 
- + ステップ 1
  - + ステップ 2
    - + サブステップ

/ **Typst**: モダンな組版言語  
/ **Touying**: スライド作成パッケージ

## 1. サブステップ

## 3. ステップ3

## 用語リスト

Typst モダンな組版言語

Touying スライド作成パッケージ

## コードブロック

コードブロック

Python の例 (シンタックスハイライト)

```
def fibonacci(n: int) → list[int]:  
    """フィボナッチ数列を生成する"""  
    result = [0, 1]  
    for i in range(2, n):  
        result.append(result[-1] + result[-2])  
    return result  
  
print(fibonacci(10))  
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

## コードブロック (複数言語)

コードブロック (複数言語)

Rust

```
fn main() {  
    let msg = "Hello,  
Typst!";  
    println!("{}", msg);  
}
```

Shell

```
$ typst compile main.typ  
$ typst watch main.typ
```

JavaScript

```
const greet = (name) => {  
    return `Hello, ${name}!`;  
};  
console.log(greet("World"));
```

SQL

```
SELECT name, age  
FROM users
```

```
WHERE age ≥ 20  
ORDER BY name;
```



## 画像の挿入

画像の挿入

**#image** で直接挿入



```
#image("assets/example.png",  
width: 80%)
```

**#figure** でキャプション付き



Figure 1: サンプル画像

```
#figure(  
  image("path/to/img.png",
```

```
width: 60%),  
    caption: [キャプション],  
)
```

# グリッドレイアウト

グリッドレイアウト

列 1

**1fr** で均等分割

列 2

**1fr** で均等分割

列 3

**1fr** で均等分割

```
#grid(  
  columns: (1fr, 1fr, 1fr), // 列幅指定 (固定値も可: 200pt)  
  gutter: 1em, // 隙間  
  [列1の内容], [列2の内容], [列3の内容],  
)
```

# ボックス・装飾

## ボックス・装飾

### 警告ボックス

**#rect** で背景色・枠線・角丸を指定

**#block** はインライン化しない矩形領域

```
#rect(  
  fill: yellow.lighten(60%),  
  stroke: orange,  
  inset: 1em,  
  radius: 8pt,  
  width: 100%,  
)[内容]
```

```
#block(  
  fill: luma(230),  
  inset: 0.8em,
```

```
radius: 4pt,  
)[内容]
```

# 数式

数式

インライン数式:  $E = mc^2$ 、  
 $a^2 + b^2 = c^2$

ブロック数式

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

// インライン

$E = m c^2$

// ブロック

$\sum_{k=1}^n k = (n(n+1))/2$

$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$

$\text{mat}(a, b; c, d) \text{vec}(x, y)$

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax + by \\ cx + dy \end{pmatrix}$$

$$= \text{vec}(a \ x + b \ y, \ c \ x + d \ y) \ \$$$

# テーブル

テーブル

| 言語     | 用途                    | 人気度   |
|--------|-----------------------|-------|
| Python | AI /<br>デー<br>タ分<br>析 | ★★★★★ |

```
#table(  
  columns: (auto, 1fr,  
  auto),  
  inset: 0.6em,  
  align: (left, left,  
right),  
  table.header(  
    [*言語*], [*用途*], [*人気  
*],  
  ),  
  [Python], [AI], [★★★★★],
```



| 言語    | 用途                 | 人気度   |
|-------|--------------------|-------|
| Rust  | システム<br>開発         | ★★★★☆ |
| Typst | 組<br>版・<br>ド<br>キュ | ★★★★☆ |

) [Rust], [システム], [★★★★☆],

| 言語 | 用途  | 人気度 |
|----|-----|-----|
|    | メント |     |

## アニメーション — **#pause**

アニメーション — **#pause**

ステップごとに内容を表示できます：

1. まず最初のポイントが表示される
- 2.
- 3.

## アニメーション — **#pause**

アニメーション — **#pause**

ステップごとに内容を表示できます：

1. まず最初のポイントが表示される
2. 次にこのポイントが現れる
- 3.

## アニメーション — **#pause**

アニメーション — **#pause**

ステップごとに内容を表示できます：

1. まず最初のポイントが表示される
2. 次にこのポイントが現れる
3. 最後にこれが表示される

## アニメーション — **#pause**

アニメーション — **#pause**

ステップごとに内容を表示できます：

1. まず最初のポイントが表示される
2. 次にこのポイントが現れる
3. 最後にこれが表示される

**#pause** を使うと、その位置でスライドが分割されます。

## アニメーション — **#only** / **#uncover**

アニメーション — **#only** / **#uncover**

- **#pause** — 以降を次のステップへ
- **#only(n)[ ... ]** — n 番目のステップのみ表示
- **#uncover(n)[ ... ]** — n 番目で可視化（スペースは保持）

**#only(1)**[ステップ1だけ表示]

**#only(2)**[ステップ2だけ表示]

ステップ 1: 最初の状態

// 範囲指定

#**uncover**( "2-" ) [2ステップ目以降]

#**only**( "1-2" ) [1~2ステップ目]



## アニメーション — **#only** / **#uncover**

アニメーション — **#only** / **#uncover**

- **#pause** — 以降を次のステップへ
- **#only(n)[ ... ]** — n 番目のステップのみ表示
- **#uncover(n)[ ... ]** — n 番目で可視化（スペースは保持）

**#only**(1)[ステップ1だけ表示]

**#only**(2)[ステップ2だけ表示]

ステップ 2: 変化した状態

// 範囲指定

#**uncover**( "2-" ) [2ステップ目以降]

#**only**( "1-2" ) [1~2ステップ目]

## リンク・引用

リンク・引用

### リンク

Typst 公式サイト

URL そのまま: `https://github.com/touying-typ/touying`

### ブロック引用

```
#link("https://typst.app")[表示テキスト]
```

// URLをそのまま表示しつつリンクにする

```
#link("https://example.com")
```

```
#quote(  
  block: true,  
  attribution: [著者名],  
)[
```

Beware of bugs in the  
above code;

I have only proved it  
correct, not tried it.

— Donald Knuth

引用文をここに書く

]

# Typst スクリプト（変数・関数・制御構造）

Typst スクリプト（変数・関数・制御構造）

変数: Typst v0.12

こんにちは、Touying さん！

良好

アイテム 1   アイテム 2   アイテ  
ム 3

```
#let title = "Typst"  
#let version = 0.12  
変数: *#title* v#version
```

```
#let greet(name) = [  
  こんにちは、*#name* さん！  
]  
#greet("Touying")
```

```
#let score = 85  
#if score ≥ 90 [優秀]
```

```
else if score ≥ 70 [良好]  
else [要努力]
```

```
#for i in range(1, 4) [  
    アイテム #i  
]
```

## #show / #set ルール

#show / #set ルール

#set でデフォルト設定を変更:

// フォント設定

```
#set text(font: "Noto Sans  
JP", size: 12pt)
```

// 段落設定

```
#set par(justify: true,  
leading: 0.8em)
```

// 見出し番号なし

### 使いどころ

- #set → 以降の要素に継続適用
- #show → 特定要素を丸ごと変換
- スライドの冒頭に書くことでグローバル設定に

```
#set heading(numbering:  
none)
```

#show で要素を変換:

```
// raw テキストのフォント変更  
#show raw: set text(font:  
"Geist Mono")
```

```
// 強調を赤色に  
#show strong: set text(fill:  
red)
```



# 整列・スペーシング

整列・スペーシング

← 左寄せ

中央寄せ

右寄せ →

center + horizon で  
縦横中央に配置

`#align(left)`[左寄せ]

`#align(center)`[中央寄せ]

`#align(right)`[右寄せ]

`#v(1em)` // 縦方向スペース

`#h(1em)` // 横方向スペース

`#align(center + horizon)`[  
縦横中央  
]

# 文法チートシート まとめ

## 文法チートシート まとめ

| カテゴリ | 主な構文                                                                                                                       |
|------|----------------------------------------------------------------------------------------------------------------------------|
| テキスト | <b>*太字*</b> <i>_斜体_</i> <code>code</code> <code>#text(fill:)</code> <code>#underline[]</code><br><code>#highlight[]</code> |
| リスト  | - 箇条書き、+ 番号付き、/ <b>用語</b> : 定義リスト                                                                                          |
| コード  | ... ブロック、 ... インライン                                                                                                        |
| 画像   | <code>#image("path", width:)</code> , <code>#figure(image( ... ),</code><br><code>caption:[])</code>                       |

| カテゴリ    | 主な構文                                                                   |
|---------|------------------------------------------------------------------------|
| グリッド    | <code>#grid(columns:, gutter:, [ ... ], [ ... ])</code>                |
| ボックス    | <code>#rect(fill:, stroke:, inset:, radius:),<br/>#block( ... )</code> |
| 数式      | <code>\$ ... \$</code> インライン、 <code>\$ ... \$</code> ブロック              |
| テーブル    | <code>#table(columns:, table.header( ... ), [セル], ... )</code>         |
| アニメーション | <code>#pause, #only(n)[ ... ], #uncover("n-")[ ... ]</code>            |

| カテゴリ   | 主な構文                                                                      |
|--------|---------------------------------------------------------------------------|
| リンク・引用 | <code>#link("url")[text], #quote(block: true, attribution:)[ ... ]</code> |
| スクリプト  | <code>#let, #if, #for, #show, #set</code> , 関数定義                          |
| 整列     | <code>#align(center + horizon)[ ... ], #v(), #h()</code>                  |